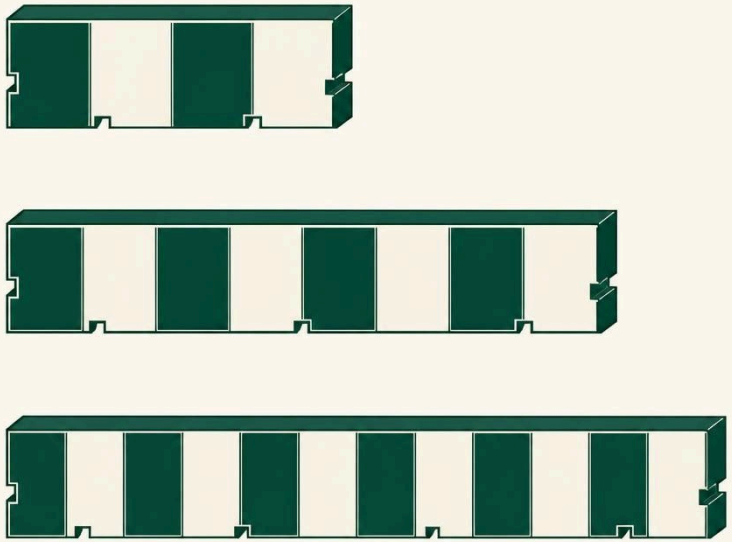
Array Dinamici in C

Il vettore dinamico, operazioni sul vettore, vettori in pratica

LINGUAGGIO C | MEMORIA DINAMICA | ARRAY DINAMICI

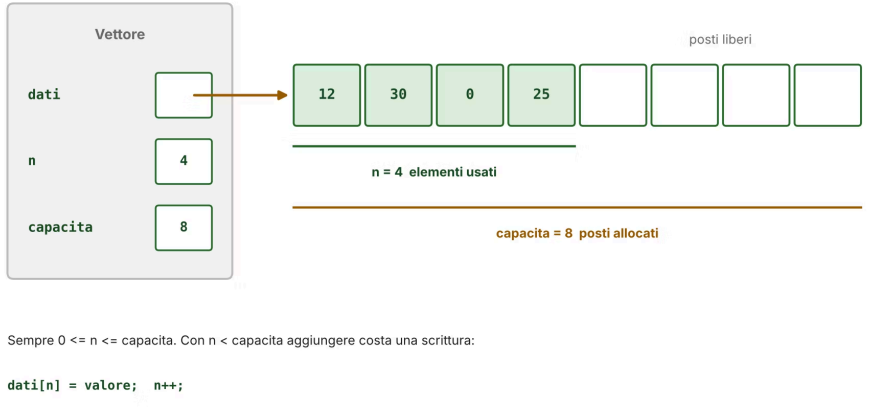


Lunghezza e Capacità

Un vettore dinamico tiene insieme tre informazioni: il blocco nell'heap, quanti elementi sono usati e quanti posti sono allocati.

```
typedef struct {
    int *dati;       /* blocco nell'heap */
    int n;           /* elementi usati (lunghezza) */
    int capacita;    /* posti allocati */
} Vettore;

v->dati[v->n] = valore;    /* se n < capacita */
v->n++;
```

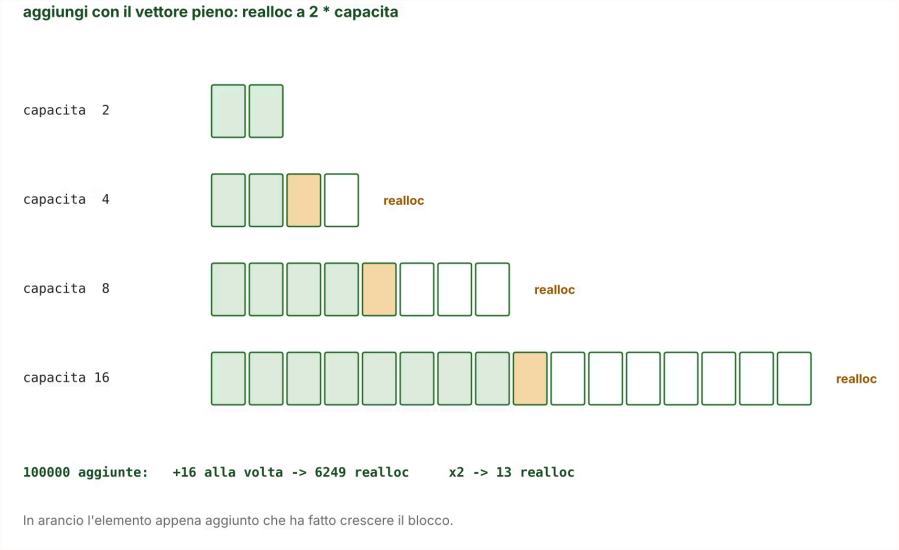


> Vale sempre 0 <= n <= capacita. Finché c'è posto, aggiungere costa una sola scrittura; i posti dopo n sono allocati ma non ancora usati.

Crescita a Raddoppio

Quando il vettore è pieno, aggiungi chiama realloc e raddoppia la capacità.

```
if (v->n == v->capacita) {
    tmp = realloc(v->dati, 2 * v->capacita * sizeof *tmp);
    if (tmp == NULL) {
        ok = 0;              /* il vettore resta com'era */
    } else {
        v->dati = tmp;
        v->capacita = 2 * v->capacita;
    }
}
```



Per 100000 aggiunte: 6249 realloc crescendo di 16 alla volta, 13 raddoppiando. Con il raddoppio gli elementi copiati sono meno di 2n.

Le Funzioni del Tipo

```
Vettore *crea_vettore(int capacita);
int aggiungi(Vettore *v, int valore);
int lunghezza(const Vettore *v);
int capacita(const Vettore *v);
int elemento(const Vettore *v, int i);
void distruggi_vettore(Vettore *v);

for (i = 0; i < lunghezza(pagine); i++) {
    totale += elemento(pagine, i);
}
```

Il main usa il vettore solo con le sue funzioni, mai con i campi: così n e capacita non possono diventare incoerenti, e cambiare la strategia di crescita tocca una sola funzione.

Funzione	Compito
crea_vettore	struct e blocco, NULL se manca memoria
aggiungi	in fondo, raddoppia se pieno
elemento	copia o puntatore all'elemento i
distruggi_vettore	libera il blocco, poi la struct

> Mai capacità iniziale 0: 2 * 0 = 0 e il vettore non cresce.

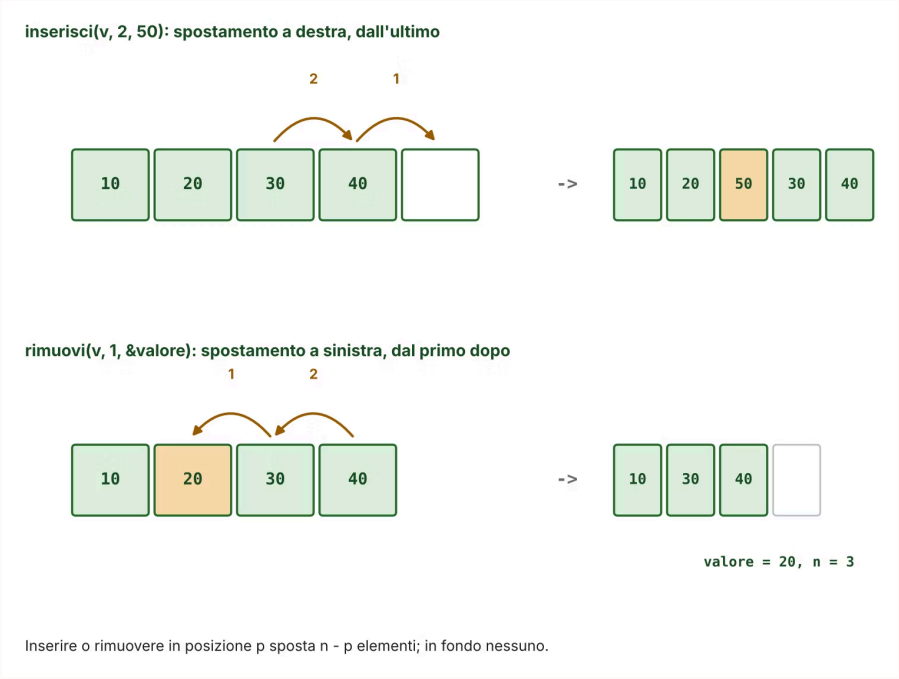
Inserire in Mezzo

Per inserire in posizione p si fa spazio spostando a destra gli elementi da p in poi, partendo dall'ultimo.

```
int inserisci(Vettore *v, int posizione, int valore);

/* dopo l'eventuale raddoppio */
for (i = v->n - 1; i >= posizione; i--) {
    v->dati[i + 1] = v->dati[i];
}
v->dati[posizione] = valore;
v->n++;
```

La posizione va da 0 (in testa) a n (in fondo). Partendo dal primo, ogni elemento sovrascriverebbe il successivo prima di spostarlo.



Rimuovere e Ridurre la Capacità

```c
int rimuovi(Vettore *v, int posizione, int *valore)
{
    int ok = posizione >= 0 && posizione < v->n;
    int i;

    if (ok) {
        *valore = v->dati[posizione];
        for (i = posizione; i < v->n - 1; i++) {
            v->dati[i] = v->dati[i + 1];
        }
        v->n--;
    }
    return ok;
}
```

togli_ultimo non sposta niente. Inserire o rimuovere in posizione p sposta n - p elementi.

Regola per dimezzare	realloc in 40 operazioni
n = capacita / 2	40 (altalena)
n = capacita / 4	0

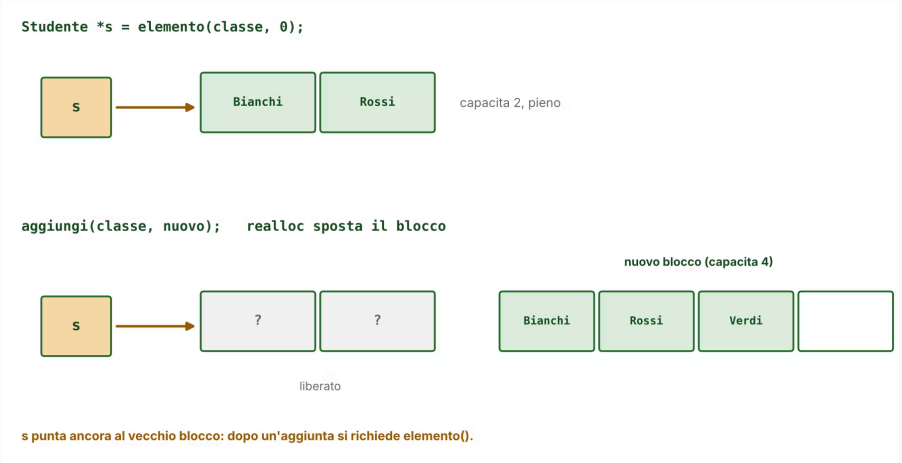
> Dimezzando a metà il vettore resta pieno e la prossima aggiunta raddoppia di nuovo: si dimezza solo quando n scende a un quarto.

Puntatore a un Elemento

Per modificare una struct dentro il vettore, elemento restituisce un puntatore. Ma punta dentro il blocco dati: se aggiungi chiama realloc, il blocco può spostarsi.

```
Studente *elemento(Classe *c, int i)
{
    return &c->dati[i];
}

Studente *s = elemento(classe, 0);
s->assenze++;              /* corretto */
aggiungi(classe, nuovo);   /* può spostare il blocco */
s->assenze++;              /* SBAGLIATO */
```



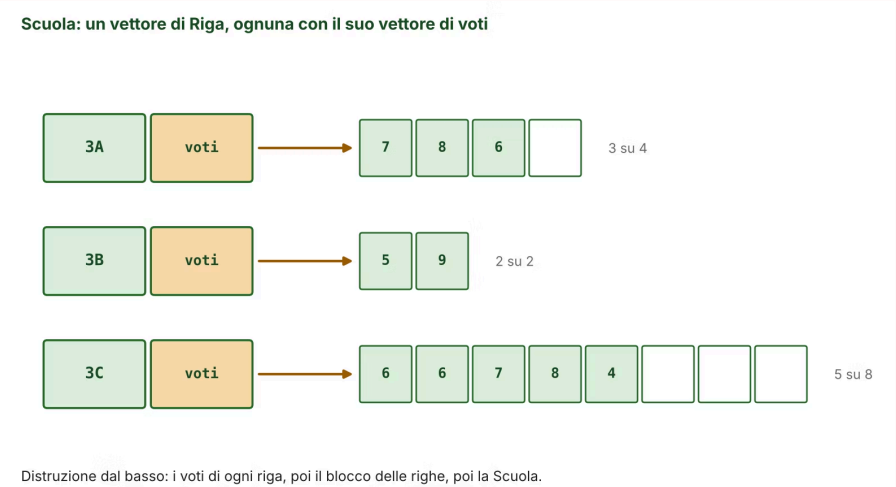
> Il puntatore restituito da elemento vale solo fino alla prossima aggiunta; dopo si richiede.

Vettore di Vettori

Quando le righe hanno lunghezze diverse, ogni riga contiene il puntatore a un proprio vettore dinamico, che cresce senza toccare gli altri.

```
typedef struct {
    char classe[8];
    Voti *voti;          /* vettore dinamico di int */
} Riga;

typedef struct {
    Riga *dati;
    int n;
    int capacita;
} Scuola;
```



> Si distrugge dal basso: prima il vettore di voti di ogni riga, poi il blocco delle righe, poi la Scuola.

Copiare e Confrontare

```
Vettore *alias = originale;      /* stesso vettore */

Vettore *copia(const Vettore *v)
{
   Vettore *c = crea_vettore(v->capacita);
   int i;

   if (c != NULL) {
      for (i = 0; i < v->n; i++) {
         c->dati[i] = v->dati[i];
      }
      c->n = v->n;
   }
   return c;
}
```

Operazione	Risultato
b = a (puntatori)	due nomi, un vettore
copia della struct	stesso blocco dati
copia(a)	struct e blocco nuovi
a == b	confronta gli indirizzi
prima_differenza(a, b)	-1 se il contenuto è uguale

> Un alias si distrugge una volta sola; una copia profonda è indipendente e va distrutta anche lei.

Funzioni che Restituiscono un Vettore

```
Storico *aumenti(const Storico *s)
{
    Storico *risultato = crea_storico(2);
    int ok = risultato != NULL;
    int i;

    for (i = 1; i < lunghezza(s) && ok; i++) {
        if (elemento(s, i).prezzo > elemento(s, i - 1).prezzo) {
            ok = aggiungi(risultato, elemento(s, i));
        }
    }
    if (!ok) {
        distruggi_storico(risultato);
        risultato = NULL;
    }
    return risultato;
}
```

Quando un risultato ha una lunghezza non nota in anticipo (i mesi di aumento, gli articoli sotto scorta, l'unione di due elenchi) la funzione crea un vettore nuovo e lo restituisce.

Se manca memoria a metà, la funzione distrugge quello che ha costruito e restituisce NULL.

> Chi chiama deve distruggere il vettore restituito: stampa(sotto_scorta(m)) perde il vettore creato.

Errori Comuni da Evitare

1

Scorrere fino a capacita invece che fino alla lunghezza.

2

Scrivere v->dati = realloc(v->dati, ...) e perdere il blocco.

3

Aggiornare la capacità prima di sapere se realloc è riuscita.

4

Spostare a destra partendo dal primo elemento.

5

Dimenticare n++ o n-- dopo inserimento e rimozione.

6

Tenere un puntatore a un elemento attraverso un'aggiunta.

7

Liberare il blocco delle righe prima dei vettori di ogni riga.

8

Credere che copiare la struct copi anche gli elementi.

Riepilogo Concetti

Situazione	Tecnica	Note
Array che cresce	struct {dati, n, capacita}	0 <= n <= capacita
Vettore pieno	realloc a 2 * capacita	costo medio costante
Uso dal main	solo le funzioni del tipo	mai i campi
Inserire in posizione p	spostare a destra dall'ultimo	n - p spostamenti
Rimuovere in posizione p	spostare a sinistra dal primo dopo	n - p spostamenti
Togliere dal fondo	togli_ultimo	nessuno spostamento
Restituire memoria	dimezzare a capacita / 4	niente altalena
Modificare un elemento	puntatore da elemento()	fino alla prossima aggiunta
Righe di lunghezza diversa	vettore di vettori	distruzione dal basso
Copia indipendente	copia profonda	blocco nuovo
Risultato di lunghezza ignota	funzione che crea un vettore	lo distrugge chi chiama

SEZIONE 1

Esempi Pratici: il Vettore Dinamico

Concetti base — tre file su lunghezza, capacità e funzioni.

1 – 010-lunghezza-e-capacita.c

Problema

i numeri di maglia della squadra di pallavolo.

Tecnica

lunghezza, capacità e posti liberi.

2 – 020-crescita-a-raddoppio.c

Problema

1000, 10000 e 100000 aggiunte.

Tecnica

+16 contro x2, realloc e copie.

3 – 030-funzioni-del-vettore.c

Problema

le pagine lette ogni giorno.

Tecnica

il vettore usato solo con le sue funzioni.

SEZIONE 1

Esempi Pratici: Problemi sul Vettore Dinamico

040 – Registro dei ritardi

Gli ingressi in ritardo dell'anno, per mese e per studente.

050 – Misure continue

Il rumore in mensa fino a STOP, con la media mobile.

060 – Numeri del lotto

Archivio di estrazioni: frequenze e ritardi.

070 – Scontrini del bar

Le vendite della giornata e il riepilogo per prodotto.

080 – Vettore di parole

Il glossario: ogni termine nel suo blocco.

SEZIONE 2

Esempi Pratici: Operazioni sul Vettore

Concetti base — tre file su inserire, rimuovere e ridurre.

1 – 010-inserire-in-mezzo.c

Problema

l'ordine delle interrogazioni.

Tecnica

inserisci con spostamento a destra.

2 – 020-rimuovere.c

Problema

i libri in prestito alla classe.

Tecnica

rimuovi e togli_ultimo.

3 – 030-ridurre-la-capacita.c

Problema

un vettore che si svuota e si riempie.

Tecnica

capacita / 4 contro capacita / 2.

SEZIONE 2

Esempi Pratici: Problemi sulle Operazioni

040 – Classifica in ordine

La corsa campestre: ogni tempo nella sua posizione.

050 – Coda della mensa

Si entra in fondo, si esce dall'inizio: il costo degli spostamenti.

060 – Annulla l'ultima operazione

Le correzioni di un voto annullate dall'ultima.

070 – Elenco senza doppioni

I volontari dell'open day: aggiungi se manca, rimuovi per valore.

080 – Unione e intersezione

Due laboratori: vettori ordinati in una sola passata.

SEZIONE 3

Esempi Pratici: Vettori in Pratica

Concetti base — tre file su struct, righe e copie.

1 – 010-vettore-di-struct.c

Problema

il registro delle assenze.

Tecnica

puntatore valido fino alla prossima aggiunta.

2 – 020-vettore-di-vettori.c

Problema

una verifica comune a più classi.

Tecnica

un vettore di voti per ogni classe.

3 – 030-copiare-e-confrontare.c

Problema

i punteggi di un gioco.

Tecnica

alias, copia profonda e confronto.

SEZIONE 3

Esempi Pratici: Problemi sui Vettori in Pratica

040 – Rubrica della classe

Aggiunta, ricerca, modifica e cancellazione dei contatti.

050 – Orario dei docenti

Un vettore di ore per docente e le ore libere comuni.

060 – Storico dei prezzi

Il prezzo del pasto in mensa e i mesi di aumento.

070 – Magazzino del laboratorio

Carico, scarico e articoli sotto scorta.

080 – Statistiche del torneo

Squadre con il proprio vettore di risultati e classifica.